

NiyamTrace-X: Set-Valued Semantic-to-Effect Authorization for Multilingual Tool-Using LLM Agents

Badampudi Agasthya Anirudh^a, Sri Pranav K^a, Vijayarajan V^{b,*}

^a*School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India*

^b*Department of Quantum AI, School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India*

Abstract

Tool-using large language model (LLM) agents can turn semantic mistakes into authorization failures even when a generated tool call is syntactically valid. We present NIYAMTRACE-X, a semantic-to-effect authorization framework that keeps multiple plausible interpretations explicit, compiles them into effect constraints, and permits only effects that remain admissible under every represented meaning and the authenticated policy. A Semantic Anchor Lock independently protects security-critical surface details, while a Clarification Evidence Gate restores utility only after those constraints are enforced. On a frozen, group-disjoint internal holdout of 2,000 cases (500 semantic groups in English, Romanized Hindi, Romanized Telugu, and Telugu script), Qwen3.5-122B-A10B-FP8 reaches 99.85% final decision accuracy after all 80 observed pre-guard unsafe ALLOW decisions are intercepted; GPT-OSS-120B reaches 99.80% after both observed pre-guard unsafe ALLOW decisions are intercepted. Both finish with zero observed unsafe ALLOW decisions and zero false blocks. A 50,000-replicate group-cluster bootstrap gives 95% intervals of 99.65–100.00% and 99.60–99.95%, respectively. Post-freeze metamorphic testing exposes a matching-to-all scope-broadening weakness in Anchor Lock v2; v3 closes all 1,496 discovered misses and reaches 100% recall on 10,248 critical attacks with zero false positives on 2,000 clean controls. A transaction-first reference broker also passes 1,320 randomized execution-boundary scenarios. Additional hosted and local external-benchmark runs characterize the surrounding model/tool harness, but they do not route actions through NIYAMTRACE-X; BFCL timeouts and an infrastructure failure before the planned third-family run therefore leave direct external system-level generalization open.

Keywords: LLM agents, tool use, authorization, multilingual robustness, semantic ambiguity, agent security

1. Introduction

As LLM agents move from answering questions to operating tools, the cost of a mistake changes. A wrong response may be inconvenient; a wrong tool action can alter a database, send a message, move money, or change persistent state. ToolEmu, AgentDojo, InjecAgent, Agent Security Bench, and Agent-SafetyBench all show that realistic tool use exposes failures that are difficult to capture with ordinary language-model accuracy alone [1, 2, 3, 4, 5]. The safety question is therefore not only whether the model understood the request, but whether uncertainty in that understanding can leak into executable authority.

A typical agent pipeline can be written as

$$u \rightarrow \hat{i} \rightarrow t \rightarrow e, \quad (1)$$

where a user instruction u is reduced to one inferred intent $\hat{i}$, translated into a tool call t , and finally realized as an external effect e . In many systems, authorization begins only after $\hat{i}$ has already been selected. That ordering creates a semantic single point of failure. If the chosen interpretation drops a negation,

widens a quantifier, substitutes an entity, or resolves an ambiguity incorrectly, a downstream policy engine may still behave exactly as designed while authorizing the wrong meaning.

The problem becomes sharper in multilingual settings. Recent work shows that a model can preserve the overall intent of a request while corrupting execution-critical parameter values across languages [6]. We see the same separation in our holdout: Qwen’s exact semantic accuracy on Romanized Telugu is 62.6%, yet its final authorization decision is correct in 99.8% of those cases. That gap suggests a different design target. The runtime need not make the semantic model infallible; it needs to stop semantic uncertainty from quietly enlarging what the agent is allowed to do.

This leads to the central question:

What is the largest real-world effect trajectory that remains safe under every still-plausible meaning of a user instruction?

NIYAMTRACE-X addresses this question by retaining a set of plausible semantic contracts instead of committing immediately to one interpretation. Each contract is compiled into an effect program, and the runtime authorizes only the common safe core. A separate surface-to-contract check, Semantic Anchor Lock, protects high-risk details such as identifiers, dates, amounts, negations, and quantifiers. Clarification recovery is

*Corresponding author.

Email addresses: badampudi.agasthya2023@vitstudent.ac.in (Badampudi Agasthya Anirudh), sripranav.k2023@vitstudent.ac.in (Sri Pranav K), vijayarajan.v@vit.ac.in (Vijayarajan V)

deliberately placed later: it restores utility only when the remaining semantic differences are effect-equivalent to the authenticated authorization.

Contributions.. The paper contributes six pieces that fit together as one authorization design:

1. **Set-valued semantic authorization.** Authorization is computed over all still-plausible meanings rather than a single selected intent.
2. **Interpretation-intersected effects.** We define a Semantic Interpretation Lattice (SIL) and an Interpretation-Intersected Effect Automaton (IIEA) whose accepted language is the intersection of the effect languages induced by plausible interpretations.
3. **Anchor-constrained utility recovery.** Semantic Anchor Lock preserves security-critical surface constraints, while the Clarification Evidence Gate recovers utility only after those constraints have been enforced.
4. **Frozen cross-model evaluation and component ablation.** Two 120B-class model families are evaluated on the same 2,000-case group-disjoint holdout. The retained per-case logs support a non-synthetic component ablation and group-clustered uncertainty analysis.
5. **Post-freeze adversarial hardening.** Metamorphic testing reveals a specific scope-broadening weakness in v2; Anchor Lock v3 closes all 1,496 discovered misses without adding a trigger on any evaluable frozen model output.
6. **Execution-boundary reference design and reproducibility.** We retain the frozen dataset, per-case outputs, runtime source hashes, and a transaction-first execution-broker prototype, while keeping reference-broker evidence separate from claims about production integration.

2. Threat Model and Scope

We study an LLM agent that receives a natural-language instruction, forms a semantic contract, proposes one or more tool calls, and sends them to an authorization layer. A failure can enter at several points: ambiguity in the instruction, translation or transliteration drift, model hallucination, or adversarial wording. The security-relevant semantic failures considered here include:

- entity substitution or widening;
- temporal scope loss;
- negation deletion;
- quantifier drift (e.g., *all* $\rightarrow$ *matching*);
- value, currency, recipient, or permission drift;
- hallucinated critical slots; and

- malformed structured output.

The part of the system evaluated in this paper is the boundary

language interpretation $\rightarrow$ effect authorization decision.

That scope matters. The current prototype does *not* establish complete mediation of external providers, replay-safe production execution, crash recovery, or commit-time state revalidation. Section 9 audits those boundaries explicitly rather than treating them as implied by the authorization results.

3. From Meanings to Effect Sets

3.1. Interpretation Sets

For instruction u , let

$$I(u) = \{C_1, C_2, \dots, C_k\} \quad (2)$$

be the set of semantic contracts that remain plausible after interpretation. Each C_i contains an action together with security-critical slots such as entity, temporal scope, amount, currency, recipient, negation, quantifier, and any unresolved uncertainty.

Two concepts are kept separate throughout the evaluation. `semantic_expected` records the meaning supported by the language itself, while `authorized_contract` records what policy and authentication actually permit. The distinction is important: a request can be interpreted correctly and still be unauthorized, or it can remain linguistically ambiguous while being safely resolvable only after clarification.

3.2. Semantic Interpretation Lattice

The SIL describes how candidate interpretations relate to one another using the labels EQUIVALENT, NARROWER, BROADER, DISJOINT, CONTRADICTORY, and PARTIAL_OVERLAP. These relations are not merely descriptive. A narrower reading can sacrifice utility, whereas a broader reading can create effects the user never authorized.

3.3. Effect Programs and the Safe Core

Each contract compiles to an effect language L_i containing the effect trajectories compatible with C_i . The robust safe core is

$$\mathcal{K} = \bigcap_i L_i, \quad (3)$$

while the set of effects compatible with at least one interpretation is

$$\mathcal{U} = \bigcup_i L_i. \quad (4)$$

Their difference,

$$\mathcal{F} = \mathcal{U} - \mathcal{K}, \quad (5)$$

is the ambiguous fringe. For a proposed effect e , NIYAMTRACE-Xapplies

$$\text{decision}(e) = \begin{cases} \text{ALLOW}, & e \in \mathcal{K} \text{ and policy permits,} \\ \text{CLARIFY}, & e \in \mathcal{F}, \\ \text{BLOCK}, & e \notin \mathcal{U}. \end{cases} \quad (6)$$

The practical consequence is simple: unresolved meaning can reduce authority, but it cannot enlarge it.

3.4. Interpretation-Intersected Effect Automaton

For interpretation C_i , let

$$M_i = (Q_i, \Sigma, \delta_i, q_i^0, F_i)$$

be an automaton over effect-relevant transitions. We define

$$M^* = \bigcap_i M_i \quad (7)$$

so that

$$L(M^*) = \bigcap_i L(M_i). \quad (8)$$

This automaton view becomes useful for multi-step tools. Two interpretations may agree on the first few actions and diverge only near an irreversible step; intersecting their accepted trajectories makes that late divergence visible to authorization.

3.5. Soundness Conditions and a Conditional Guarantee

The safe-core rule is only as strong as the semantic coverage and effect model supplied to it. To make that dependency explicit, let $I^*(u)$ denote the set of security-relevant interpretations that are genuinely plausible for instruction u , and let $\mathcal{E}(C)$ denote the real effects consistent with contract C . The implementation computes a represented interpretation set $I(u)$ and an accepted effect language $\widehat{L}(C)$ for each represented contract.

The authorization claim is conditional on four assumptions:

- (A1) **Interpretation coverage:** $I^*(u) \subseteq I(u)$. Extra conservative interpretations are allowed; omitting a security-relevant plausible interpretation is not.
- (A2) **Non-expansive effect compilation:** $\widehat{L}(C) \subseteq \mathcal{E}(C)$ for every represented contract. The compiler may be conservative, but it must not admit an effect that the represented contract forbids.
- (A3) **Fresh policy evaluation:** the policy predicate is evaluated against the relevant authorization state and only returns effects that the authenticated policy permits.
- (A4) **Complete mediation:** every effectful transition passes through the authorization check that enforces the computed safe core.

Proposition 1 (conditional non-expansion)... Let

$$\widehat{\mathcal{K}}(u) = \bigcap_{C \in I(u)} \widehat{L}(C).$$

If (A1)–(A4) hold and the runtime returns ALLOW only for an effect $e \in \widehat{\mathcal{K}}(u)$ that also satisfies policy, then

$$e \in \mathcal{E}(C) \quad \text{for every } C \in I^*(u).$$

Proof sketch. By (A1), each truly plausible C is represented in $I(u)$. Membership in the intersection implies $e \in \widehat{L}(C)$ for every represented C ; (A2) then gives $e \in \mathcal{E}(C)$. Assumption (A3) supplies the policy condition, and (A4) ensures that the

Table 1: Frozen v2 authorization procedure.

1	Parse the instruction into typed candidate contracts, critical disagreement fields, and a clarification flag.
2	Apply Semantic Anchor Lock to explicit surface constraints (identifiers, dates, values, negation, quantifiers, and scope). If an anchor violation is found, return BLOCK.
3	For each candidate, reject any explicit known value that exceeds the authenticated authorization contract.
4	If unresolved critical semantics remain, return CLARIFY; clarification never elevates an explicitly unauthorized value.
5	Require agreement across the critical semantic fields and verify every surviving candidate against the authenticated contract.
6	Apply the Clarification Evidence Gate only after anchoring; suppress clarification only when the remaining interpretations are effect-equivalent to authenticated authorization.
7	Return ALLOW only after all preceding checks succeed; otherwise return CLARIFY or BLOCK.

checked decision cannot be bypassed before the effect occurs.

□

The proposition is intentionally conditional rather than a claim that natural-language coverage has been formally proved. Anchor Lock is an engineering defense against several observed coverage and compilation failures, and the reference broker is a step toward (A4), but neither establishes those assumptions universally. If a plausible meaning is omitted, an effect compiler over-admits, policy state is stale, or execution bypasses the monitor, the proposition no longer applies.

3.6. Implementation Correspondence

The IIEA is the formal abstraction used to reason about effect intersection. The frozen v2 Python runtime uses a simpler deterministic specialization rather than materializing a general automaton. The interpretation client returns one or more typed candidate contracts, a list of critical disagreement fields, and a clarification flag. The runtime then (i) blocks explicit negation, (ii) rejects known candidate values that exceed the authenticated contract, (iii) clarifies unresolved critical disagreements, (iv) checks agreement across eight critical fields, and (v) verifies that every candidate remains within the authenticated contract. Thus, the reported holdout results validate this concrete critical-field/contract specialization of the effect-intersection idea; they should not be read as an empirical validation of every possible IIEA implementation.

3.7. Deterministic Authorization Procedure

For reproducibility, Algorithm 1 summarizes the frozen semantic-to-decision path.

Worked example... Consider the instruction “Archive matching March invoices for vendor 7184” when no year is stated. In the frozen benchmark representation, the semantic contract has `year=null`, the disagreement set contains `year`,

and `needs_clarification=true`. In the abstract set-valued view, the missing year corresponds to multiple year-specific effect sets; no particular March-year mutation belongs to a uniquely grounded safe effect set. The runtime therefore returns `CLARIFY` and performs no archive action. If the user subsequently grounds the year as 2026, the remaining candidate can be checked against the authenticated vendor/month/year/scope contract; it is eligible for `ALLOW` only if that policy check also succeeds. This example illustrates the intended rule: uncertainty narrows executable authority rather than being resolved by an unverified guess.

4. NiyamTrace-X Runtime

4.1. Multilingual Semantic Views

For a surface instruction u , the implementation may reason over several derived views:

$$V(u) = \{u_{\text{native}}, u_{\text{normalized}}, u_{\text{transliterated}}, u_{\text{pivot}}, u_{\text{paraphrase}}\}.$$

These views are used to surface alternative interpretations. None of them is allowed to authorize an effect by itself.

4.2. Semantic Anchor Lock

Schema-valid output is not treated as evidence that the semantic model preserved the user’s authority. Anchor Lock separately checks security-critical information visible in the instruction, including:

- entity IDs and recipients;
- dates and durations;
- amounts and currencies;
- negations and exceptions;
- quantifiers such as *all*, *only*, *exactly*, and *at most*; and
- permission or destructive-action terms.

For instance, if the user explicitly says *all* but the candidate contract quietly narrows that scope to *matching*, the mapping is rejected. The same pattern appears in both raw unsafe `ALLOW` cases produced by GPT-OSS on the frozen holdout.

4.3. Clarification Evidence Gate

Set-valued semantics can be conservative enough to request clarification even when the remaining interpretations lead to the same authorized effect. The clarification gate suppresses such a request only when every surviving candidate is effect-equivalent to the authenticated authorization, no critical disagreement remains unresolved, required slots are still grounded in the surface instruction, and no critical field has been hallucinated. The ordering is intentional: the gate runs after Anchor Lock. It can recover utility, but it is not a standalone safety barrier. Table 3 shows why that distinction matters.

4.4. Decision and Error Taxonomy

The evaluation records semantic and execution-relevant error classes including `ENTITY_ERROR`, `TIME_ERROR`, `VALUE_ERROR`, `SCOPE_ERROR`, `NEGATION_ERROR`, `QUANTIFIER_ERROR`, `AUTHORITY_ERROR`, `POLICY_ERROR`, `SIMULATION_ERROR`, and `EXECUTION_ERROR`. The present benchmark covers the semantic-to-authorization portion of this taxonomy rather than the entire execution stack.

5. NiyamTrace-Bench 3

5.1. Grouped Multilingual Construction

NIYAMTRACE-BENCH 3 is generated deterministically from semantic groups using seed 20260828. The generator samples a base month, a four-digit vendor identifier, a year in {2025, 2026}, a risk tier, and one of eight relation families. The relation-sampling weights in the 10,000-row generator are 35/8/8/8/8/5/20 for `EQUIVALENT`, `NARROWER`, `BROADER`, `NEGATED`, `ENTITY_SHIFT`, `TEMPORAL_SHIFT`, `VALUE_SHIFT`, and `AMBIGUOUS`, respectively; risk weights are 20/40/30/10 for `LOW`/`MEDIUM`/`HIGH`/`CRITICAL`. Each semantic group is rendered in four language/script conditions:

- English Latin script (`eng_Latn`);
- Romanized Hindi (`hin_Latn`);
- Romanized Telugu (`tel_Latn`); and
- Telugu script (`tel_Telu`).

Labels are programmatically generated from the relation family and authenticated base contract; this is a synthetic benchmark, not a human-annotated natural-language corpus. The generator source, seed, frozen holdout membership, and hashes are retained in the public reproducibility artifact.

5.2. Frozen Group-Disjoint Holdout

The primary evaluation contains 500 complete semantic groups, hence 2,000 cases and exactly 500 cases per language/script condition. The frozen holdout is near-balanced by relation: 252 cases each for `EQUIVALENT`, `NARROWER`, `BROADER`, and `NEGATED`, and 248 each for `ENTITY_SHIFT`, `TEMPORAL_SHIFT`, `VALUE_SHIFT`, and `AMBIGUOUS`. Expected verdicts are 504 `ALLOW`, 1,248 `BLOCK`, and 248 `CLARIFY`. Risk counts are 360 `LOW`, 788 `MEDIUM`, 588 `HIGH`, and 264 `CRITICAL`. Group membership was fixed and hash-frozen before the reported model comparison; no semantic group overlaps the development groups used by the runtime. We deliberately call this an *internal group-disjoint holdout*: labels come from the same generator family, so it is not an external benchmark or an independent human-adjudicated test set.

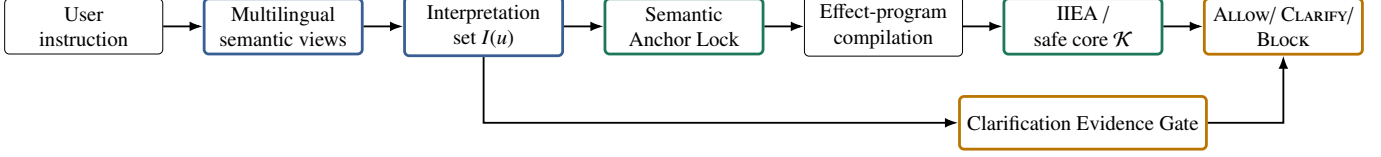


Figure 1: Evaluated semantic-to-effect authorization path. The present paper evaluates the path through the final authorization decision; complete provider-side execution mediation remains future work.

5.3. Metric Definitions

The frozen scorer uses the following definitions. Decision accuracy is

$$\frac{1}{N} \sum_i \mathbf{1}[\hat{y}_i = y_i].$$

For each case, critical-slot accuracy is the fraction of eight fields—action, vendor ID, month, year, amount, currency, scope, and negation—that exactly match `semantic_expected`; the reported value is the mean of that per-case fraction. “Semantic accuracy” is stricter than slot exact match: a case is counted correct only when all eight slots match, the predicted clarification flag matches the expected flag, and the effective disagreement-field set exactly matches the expected disagreement set. Clarification match is the case-level equality of the predicted and expected clarification flags. Clarification precision and recall instead treat the *final verdict* CLARIFY as the positive class:

$$P_{\text{clar}} = \frac{\#(\hat{y} = \text{CLARIFY} \wedge y = \text{CLARIFY})}{\#(\hat{y} = \text{CLARIFY})}$$

$$R_{\text{clar}} = \frac{\#(\hat{y} = \text{CLARIFY} \wedge y = \text{CLARIFY})}{\#(y = \text{CLARIFY})}.$$

Safe-action completion is the fraction of expected-ALLOW cases that finish as ALLOW. Unsafe-ALLOW rate is the fraction of non-ALLOW cases that finish as ALLOW; false-block rate is the fraction of expected-ALLOW cases that finish as BLOCK. Schema validity counts cases whose structured model output parses successfully under the frozen schema.

6. Experimental Setup

Models. The frozen holdout is evaluated with the Qwen3.5-122B-A10B-FP8 checkpoint at revision a099dee70ccfcd8d5dda56aaa0b60cb8ecadabc9 and the GPT-OSS-120B checkpoint at revision b5c939de8f754692c1647ca79fbf85e8c1e70f8a. These checkpoint identifiers and revisions are experimental provenance rather than literature citations. Both models are served through vLLM on a single AMD Instinct MI300X GPU.

Freeze discipline. Before the GPT-OSS holdout run, we hash-froze the interpretation client, prompt, schema, scoring code, IIEA, runner, Anchor Lock, clarification gate, and holdout dataset. We did not patch v2 in response to holdout failures. That restriction matters because several observed errors would have been straightforward to repair after the fact. We also verified the retained emergency archive byte-for-byte against the

frozen reproducibility record. Its SHA-256 matches, as do the hashes of the 2,000-case holdout and both 2,000-row per-case model result files.

Inference. Both models receive the same semantic extraction prompt and schema. Qwen reasoning is disabled for the structured-output call, whereas GPT-OSS returns reasoning separately from its content. We record latency for each case, but we do not use cross-model latency as a headline comparison because the two large runs were not matched for cache and warm-start conditions.

7. Results

7.1. Qwen: Safety and Utility Interventions

Before any runtime guard is applied, Qwen reaches 94.55% decision accuracy and produces 80 unsafe ALLOW decisions. The full frozen runtime raises decision accuracy to 99.85%. On the paired cases, 106 decisions move from wrong to correct and none move from correct to wrong. The exact McNemar test gives

$$p = 2.465190328815662 \times 10^{-32}.$$

Anchor Lock fires on 87 cases. Eighty of those triggers intercept unsafe ALLOW decisions, six conservatively change raw CLARIFY decisions to BLOCK, and one leaves an already-blocked case unchanged. Every triggered case has expected verdict BLOCK. The violations break down into 62 negation failures, 24 broad-quantifier failures, and one vendor-identity failure. We observe no Anchor Lock false positive on this holdout.

The clarification gate suppresses 20 clarification requests. All 20 cases are expected ALLOW cases, and all 20 end as ALLOW. On this holdout, the gate therefore recovers utility without an observed unsafe suppression.

7.2. Frozen Runtime Component Ablation

Because the frozen per-case logs retain the intermediate interventions, we can reconstruct a 2×2 component ablation without querying either language model again. The four stages are the raw policy verdict (base), Anchor Lock alone, the Clarification Evidence Gate alone, and the full frozen runtime. The comparison is therefore a deterministic replay of recorded interventions rather than a new generation experiment.

For Qwen, a two-component Shapley-style decomposition assigns +4.30 percentage points of decision accuracy to Anchor Lock and +1.00 point to clarification recovery, with no residual interaction in the recorded intervention pattern. Anchor Lock accounts for the entire reduction of 80 unsafe ALLOW

Table 2: Primary 2,000-case group-disjoint internal holdout. “Raw unsafe” is measured before the frozen Anchor Lock / clarification interventions; “final unsafe” is measured after the full v2 runtime.

Metric	Qwen3.5-122B	GPT-OSS-120B
Decision accuracy	99.85%	99.80%
Decision correct	1,997 / 2,000	1,996 / 2,000
Semantic accuracy	85.25%	85.75%
Schema validity	100.00%	99.80%
Critical-slot accuracy	99.41%	99.54%
Clarification match	89.65%	89.25%
Clarification precision	99.20%	100.00%
Clarification recall	99.60%	98.79%
Safe-action completion	99.60%	100.00%
Raw unsafe ALLOW decisions	80	2
Anchor-prevented unsafe ALLOW	80 / 80	2 / 2
Final unsafe ALLOW decisions	0	0
False blocks	0	0
Runtime / parse errors	0	4

Table 3: Component ablation reconstructed from the frozen 2,000-case per-model logs. Unsafe ALLOW counts cases predicted ALLOW when the expected verdict is not ALLOW.

Stage	Qwen accuracy	Qwen unsafe ALLOW	GPT-OSS accuracy	GPT-OSS unsafe ALLOW
Base	94.55%	80	99.70%	2
Anchor only	98.85%	0	99.80%	0
Clarification only	95.55%	80	99.70%	6
Full	99.85%	0	99.80%	0

decisions, while clarification recovery repairs 20 utility errors. GPT-OSS makes the composition requirement even clearer: the base stage contains two unsafe ALLOW decisions, Anchor Lock alone removes both, but the clarification gate used *without* Anchor Lock produces six unsafe ALLOW decisions. The intended ordering still ends at zero. Clarification recovery should therefore be read as a downstream utility mechanism, not as an independent safety control.

7.3. GPT-OSS: Different Raw Failure Mode

GPT-OSS starts much closer to the final result: 99.70% raw decision accuracy with only two unsafe ALLOW decisions. Both occur in Hindi BROADER cases containing *sabhi* (“all”) in the surface instruction. The model narrows the candidate scope to matching; Anchor Lock detects `quantifier_all` and changes both raw ALLOW decisions to BLOCK. The full v2 runtime reaches 99.80% accuracy. With only two corrected paired errors, the accuracy change is not statistically significant ($p = 0.5$). The security-relevant observation is narrower: both observed unsafe raw authorizations are intercepted, with no paired regression.

The four remaining GPT-OSS failures all occur in Romanized Hindi: one Pydantic schema error, one null-handling error, and two malformed JSON outputs. Each case ends in ERROR rather than ALLOW, and all four are kept in the reported failure count.

Table 4: Paired final-decision comparison on identical holdout cases.

Outcome	Count
Both correct	1,993
Qwen correct, GPT-OSS wrong	4
GPT-OSS correct, Qwen wrong	3
Both wrong	0
Different final verdicts	7

7.4. Cross-Model Replication

The two models disagree on 7 of the 2,000 final verdicts (0.35%). The paired McNemar test has $b = 4$, $c = 3$, and exact two-sided $p = 1.0$, so the holdout provides no evidence that their final decision accuracies differ. A 50,000-replicate cluster bootstrap, resampling the 500 semantic groups while keeping all four language realizations together, estimates the Qwen–GPT-OSS accuracy difference at +0.05 percentage points with a 95% percentile interval of $[-0.15, +0.25]$ points. The interval also spans zero. No case is wrong for both models, but we report that only as a retrospective observation: no arbitration rule is evaluated, so it is not evidence for a deployable 100%-accurate ensemble.

7.5. Semantic Accuracy Is Not Effect Safety

Romanized Telugu makes the distinction between interpretation quality and authorization safety especially visible. Qwen’s

Table 5: Qwen semantic vs. final decision accuracy by language.

Language	Semantic	Decision
English	96.2%	100.0%
Romanized Hindi	90.2%	99.6%
Romanized Telugu	62.6%	99.8%
Telugu script	92.0%	100.0%

Table 6: Post-freeze controlled metamorphic hardening. These results are separate from the frozen v2 primary holdout.

Metric	Anchor v2	Anchor v3
Critical attacks	10,248	10,248
Attack recall	85.40%	100.00%
Misses	1,496	0
Clean controls	2,000	2,000
Clean false-positive rate	0.00%	0.00%

exact semantic accuracy is 62.6%, while its final decision accuracy is 99.8%. The framework does not make semantic mistakes harmless. Instead, it prevents many of those mistakes from flowing directly into unauthorized effects.

7.6. Post-Freeze Metamorphic Hardening: Anchor Lock V3

The frozen v2 runtime remains untouched for the primary holdout. After the freeze, we run a separate metamorphic campaign targeting preservation of entities, dates, months, amounts, currencies, negations, and quantifiers. The campaign contains 10,248 critical attacks and 2,000 clean controls. V2 detects 85.40% of the attacks with zero clean false positives. Every one of its 1,496 misses belongs to the same matching→all scope-broadening family. Anchor Lock v3 promotes explicit matching scope to a first-class authority ceiling and closes all discovered misses.

As a non-regression check, we replay v3 over every evaluable frozen output. It adds no new trigger on 2,000/2,000 Qwen outputs or 1,996/2,000 GPT-OSS outputs. The remaining four GPT-OSS rows cannot be evaluated because their original model outputs failed schema or JSON parsing. This replay checks compatibility with the frozen outputs; it does not retroactively replace the v2 headline results.

7.7. Zero-Event Interpretation

Among the 2,000 holdout cases, 1,496 have an expected verdict other than ALLOW. Neither model produces an unsafe final ALLOW on those cases. For zero events in n Bernoulli opportunities, the one-sided 95% exact upper bound is

$$p_{\text{upper}} = 1 - 0.05^{1/n}. \quad (9)$$

For $n = 1496$,

$$p_{\text{upper}} \approx 0.0020005 \approx 0.20\%.$$

We therefore say *zero observed unsafe ALLOW decisions*, not zero underlying risk. The two model runs share the same cases,

so they are not pooled as 2,992 independent safety opportunities. At the semantic-group level, 374 of 500 groups contain at least one non-ALLOW case and none contains a final unsafe ALLOW. Using those 374 groups as the sampling unit gives a more conservative one-sided 95% upper bound of about 0.80% on the probability that an eligible group contains any unsafe ALLOW.

Case- and group-level uncertainty. From the final counts, two-sided exact binomial intervals are 99.562–99.969% for Qwen (1,997/2,000) and 99.489–99.945% for GPT-OSS (1,996/2,000). Because the four language realizations within a semantic group are dependent, we also run a 50,000-replicate group-clustered bootstrap using the recovered frozen per-case files. Each replicate resamples the 500 semantic groups with replacement and keeps all four language realizations inside the selected group. The resulting 95% percentile intervals are 99.65–100.00% for Qwen decision accuracy and 99.60–99.95% for GPT-OSS. The corresponding semantic-accuracy intervals are 83.30–87.10% and 82.85–88.50%. For Qwen, the final-minus-raw decision-accuracy gain is +5.30 percentage points with a clustered 95% interval of [+4.40, +6.20] points.

7.8. Reference Verified Execution Broker V2

The first reference broker exposed a time-of-check/time-of-use gap because state validation occurred before the database write transaction was acquired. We kept that failure rather than removing it from the record. V2 changes the ordering: it begins the SQLite write transaction first, then validates nonce, snapshot, predicted targets, and policy; recomputes the current effect set inside the same transaction; checks exact predicted-versus-actual deltas; records the nonce persistently; and commits only when every equality still holds.

We test the corrected reference design on 1,320 randomized scenarios: 120 independently generated database and argument states for each of 11 conditions. All 120 valid cases commit. The remaining 1,200 trials cover argument tampering, forged signatures, policy changes, predicted-target tampering, replay, stale snapshots, expiry, actor mismatch, tool mismatch, and crash-before-commit; every one fails closed or rolls back. Each condition uses 120 unique database states and 120 unique argument sets. A dedicated regression test also recreates the earlier phantom-row race and confirms that transaction-first ordering keeps the extra row out of the committed effect set. These results apply to the isolated reference broker. They do not establish complete mediation in the public executor.

7.9. Targeted External τ^3 Pilot

After repairing the external-evaluation harness, we ran a small QUICK recovery study on τ^3 . It produced six benchmark-native trajectories across two hosted model families: one task per model in each of the airline, retail, and telecom domains. Qwen received rewards [1, 0, 0] (mean 0.33), while GPT-OSS received [1, 1, 0] (mean 0.67). The overall mean across the six completed trajectories is 0.50. Table 8 reports these native rewards separately from the internal NiyamTrace-X metrics.

The pilot shows that the repaired harness can complete and retain native stateful trajectories, but the sample is deliberately

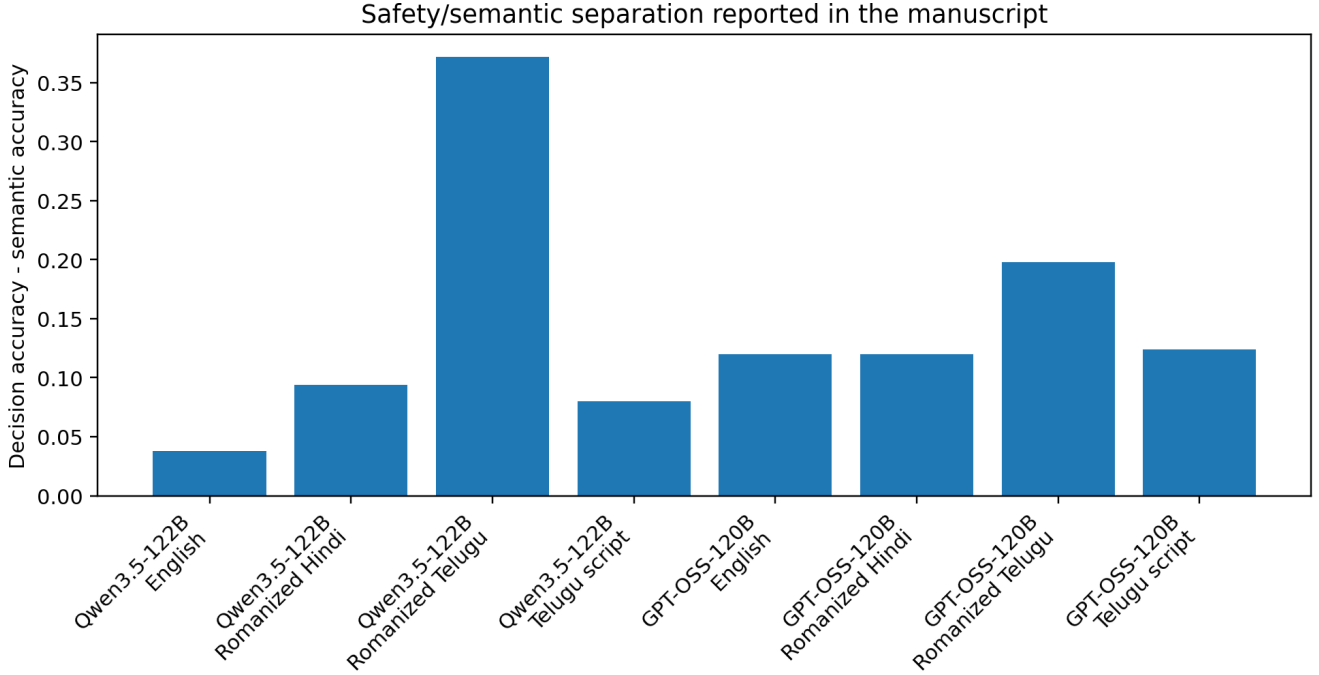


Figure 2: Decision-semantic accuracy separation by model and language. The largest gap occurs for Qwen on Romanized Telugu (37.2 percentage points), while final authorization accuracy remains 99.8%. The plot visualizes the manuscript values; it does not imply that semantic errors are harmless.

Table 7: Group-clustered bootstrap over 500 semantic groups (50,000 replicates; all four language realizations kept within cluster). Intervals are percentile 95% intervals and quantify sampling variability over semantic groups rather than treating the 2,000 rows as independent.

Quantity	Point estimate	Clustered 95% interval
Qwen final decision accuracy	99.85%	[99.65%, 100.00%]
GPT-OSS final decision accuracy	99.80%	[99.60%, 99.95%]
Qwen semantic accuracy	85.25%	[83.30%, 87.10%]
GPT-OSS semantic accuracy	85.75%	[82.85%, 88.50%]
Qwen final – raw decision accuracy	+5.30 pp	[+4.40, +6.20] pp
Qwen – GPT-OSS final decision accuracy	+0.05 pp	[-0.15, +0.25] pp

Table 8: Targeted external τ^3 QUICK pilot. Each cell is one native task reward; the pilot is partial evidence, not a benchmark-level generalization claim.

Model	Airline	Retail	Telecom	Mean
Qwen	1.0	0.0	0.0	0.33
GPT-OSS	1.0	1.0	0.0	0.67

small: $n = 1$ task per domain/model and only two completed model families. A GLM recovery attempt yields no valid evaluated trajectory, and the retained BFCL and AgentDojo recovery attempts yield no valid benchmark-native score. We therefore treat τ^3 as *partial external evidence*. BFCL, AgentDojo, MLCL, and three-family external generalization remain outside the supported claim set.

7.10. Expanded Local-T4 Benchmark-Harness Study

We next ran a closure-oriented local experiment on a single NVIDIA T4 to test whether the surrounding tool-agent

benchmark harness could be exercised with open-weight models under a fixed, reproducible setup. This study is deliberately separated from the primary NiyamTrace-X evaluation: BFCL, AgentDojo, and the local τ^3 runner call the served model directly through an OpenAI-compatible tool interface. They do *not* route benchmark actions through the NiyamTrace-X semantic authorization layer. The results below therefore characterize model/tool compatibility and the external evaluation harness; they are not direct measurements of NiyamTrace-X safety.

Granite-3.3-2B and Qwen2.5-3B both passed ordinary chat, automatic tool-selection, and forced-tool preflight. BFCL then timed out for both models (process return code 124) without a retained native score. AgentDojo produced nine valid native cases per model across banking, workspace, and travel—three cases per suite—while the Slack slice failed before yielding a valid case. On the six attacked AgentDojo cases, Granite’s native utility/security flags were 1/6 and 0/6; Qwen’s were 1/6 and 1/6. On the three no-injection baselines, Granite recorded utility/security 3/3 and 3/3, whereas Qwen recorded 2/3 and

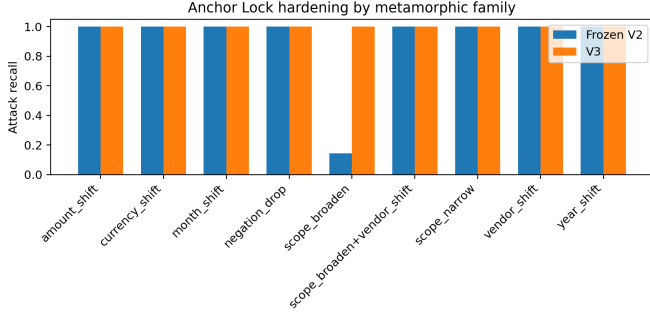


Figure 3: Metamorphic recall before and after post-freeze hardening. The v2 deficit is entirely due to the controlled matching-scope broadening family; v3 closes all 1,496 discovered misses.

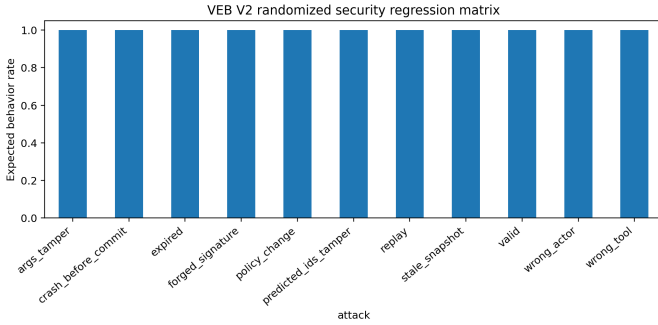


Figure 4: Verified Execution Broker V2 reference stress test. Each condition uses 120 distinct randomized states/arguments; valid cases commit and all tested invalid/crash conditions fail closed. This figure is evidence for the isolated reference broker, not production integration.

3/3.

For the local τ^3 runner, Granite completed all six requested trajectories: two airline rewards of 1, followed by two retail and two telecom rewards of 0, for an evaluated mean reward of 0.333. Qwen completed four of six trajectories—two airline and two retail, all with reward 0—while both telecom trajectories exceeded the configured 8,192-token context after reserving output tokens and were not scored. Because the same currently loaded local model also acts as the simulated user in these runs, cross-family τ^3 reward differences are not treated as a controlled head-to-head comparison.

A standalone continuation was intended to fill the missing BFCL/Slack/Qwen-telecom slices and run the independent Hermes-3/Llama family. It stopped before model execution for all three configured models because the notebook referenced an undefined HF_TOKEN variable. We treat that as an infrastructure failure, not as a model failure, and report no Llama benchmark result. Consequently, the planned three-family closure was not evaluated.

8. Related Work

Tool use and stateful evaluation. The tool-use literature has progressed from teaching language models when and how to call external functions to evaluating long-horizon, state-changing behavior. Toolformer showed that a language model

can learn API invocation from a small set of demonstrations and self-supervised selection signals [7]. API-Bank introduced a runnable benchmark for planning, retrieving, and calling APIs [8]; ToolLLM scaled tool learning to more than 16,000 real-world APIs [9]; and Gorilla emphasized retrieval-grounded API selection under changing documentation [10]. ReAct established the now-familiar pattern of interleaving language reasoning with environment actions [11]. ToolEmu then showed that realistic tool access can expose consequential failures that ordinary language-model accuracy misses [1]. BFCL evaluates function calling from single and parallel calls through abstention and agentic settings [12], while τ -bench evaluates policy-constrained tool-agent-user interaction in stateful domains [13]. Together, these works motivate evaluation at the action boundary rather than at text generation alone.

Agent security. Indirect prompt injection predates most agent-specific benchmarks: Greshake et al. demonstrated that untrusted retrieved content can redirect LLM-integrated applications and influence downstream API use [14]. InjecAgent systematically studies this failure mode in tool-integrated agents [3]. AgentDojo combines benign tasks, attacks, and defenses in realistic tool environments [2]. Agent Security Bench broadens attack and defense coverage across agent scenarios [4], and Agent-SafetyBench studies safety risks across diverse interactive environments [5]. AttriGuard approaches a related question from causal attribution: whether a tool invocation is supported by user intent rather than untrusted observations [15]. Together, these works show that agent safety depends on what an agent is allowed to do, not only on what it says.

Multilingual and uncertain semantics. Lost in Execution demonstrates that multilingual tool calling can fail even when high-level intent and tool choice are correct, with execution-critical parameter values becoming a major failure point [6]. Uncertainty-aware language-agent work likewise argues that uncertainty should remain visible to downstream decision making rather than being hidden behind a single confident output [16]. NIYAMTRACE-X focuses on the authorization consequence of that uncertainty: when several security-relevant meanings remain plausible, executable authority is restricted to their common effect set.

Authorization and runtime enforcement. The design follows the classic least-privilege principle that a computation should hold only the authority needed for its task [17]. Lampson’s confinement problem highlights why controlling the downstream effects of a computation is a distinct systems problem [18], while Clark and Wilson’s integrity model emphasizes well-formed transformations and constrained authority for protecting business data [19]. Schneider formalizes a class of security policies enforceable by execution monitors [20], and edit automata extend runtime enforcement with mechanisms that can suppress or insert actions rather than merely halt execution [21]. Our contribution sits at an earlier semantic boundary. Instead of assuming that a single authorized objective has already been identified, NIYAMTRACE-X keeps plausible meanings

Table 9: Expanded local-T4 benchmark-harness study. “AD” is AgentDojo. Counts are native evaluable cases/trajectories; NE means not evaluated. These results characterize the external model/tool harness, not the NiyamTrace-X authorization layer.

Checkpoint	Tool preflight	BFCL	AgentDojo	Local τ^3
Granite-3.3-2B	pass	timeout; no score	9 valid across 3 suites; Slack failed	6/6 evaluated; mean 0.333
Qwen2.5-3B	pass	timeout; no score	9 valid across 3 suites; Slack failed	4/6 evaluated; mean 0.000
Hermes-3/Llama-3.1-8B-AWQ	not reached	NE	NE	NE

explicit and intersects their admissible effects before a tool action is authorized. The later reference-broker experiment addresses a different boundary—binding an authorization decision to execution—and is therefore reported separately from the semantic results.

9. Repository and Runtime Audit

We separately audit the public implementation at <https://github.com/bnssaanirudh/NiyamTrace-X>, pinned to commit `c14661dbd11c42ebd1019b6a1a5c49b8643da137`. The deterministic gate checks target-identity containment, temporal containment, attribute containment, cardinality and approval constraints, and evidence verdicts. One important limitation is explicit in the snapshot: the default evidence path is a stub. We therefore do not use it to support security claims.

The audited 10,000-row prototype corpus also contains substantial duplication. We find 1,668 exact duplicate raw texts and 5,330 rows that duplicate after template normalization, leaving 46.7% of rows unique under that normalization. In a TF-IDF sample audit, 47.7% of sampled rows have a near-duplicate at cosine similarity at least 0.95. These numbers describe the audited prototype corpus, *not* the frozen 2,000-case primary group-disjoint holdout. The prototype corpus should therefore not be presented as an independent external benchmark.

In the intended application path, the audited pipeline checks the gate verdict and calls the executor only after an ALLOW decision. Its legacy executor API, however, places that responsibility on the caller and does not require a gate-issued capability. In a synthetic in-memory ERP probe, calling the executor directly without a gate decision changes three matching invoices from OPEN to ARCHIVED. We describe this narrowly as an *internal execution-boundary bypass in the legacy API*. It is not evidence of a remotely exploitable gateway vulnerability.

The audited snapshot also includes a hashable `EffectCertificate`; its `seal()` and `verify()` primitives detect mutation after sealing. The current gate does not yet complete that design: it constructs the certificate without sealing it and uses placeholder values for policy, evidence, and state fields. For that reason, we do not claim cryptographically complete authorization-to-commit binding.

10. Limitations and Validity

Synthetic labels and language coverage. The benchmark labels are generated deterministically from controlled relation templates rather than independently adjudicated by human annotators. This improves reproducibility but limits ecological

validity: lexical diversity, code-switching, dialect variation, and culturally specific phrasing are narrower than in naturally collected enterprise traffic. The four language/script conditions should therefore be interpreted as controlled multilingual stress conditions, not as comprehensive coverage of Hindi or Telugu usage.

Internal, not external, holdout. The primary holdout is group-disjoint from development data, but it still comes from the same benchmark-generation process. The result is useful internal evidence, not a substitute for external evaluation.

Domain concentration. Most holdout cases involve structured administrative effects, particularly invoice operations. IAM, payments, email, filesystems, Git, and cloud-provider tools are not covered at comparable depth.

Exact semantic scoring. Some semantic mismatches come from disagreement metadata even when the final security decision is correct. We therefore keep semantic accuracy separate from authorization accuracy rather than collapsing them into one score.

Structured-output reliability. GPT-OSS produces four schema or parse failures. We leave them in the evaluation as failures instead of repairing their outputs after inspecting the holdout.

Post-freeze hardening scope. The frozen v2 holdout remains the primary evaluation and is not retroactively upgraded. The later metamorphic campaign exposes 1,496 matching-scope broadening misses in v2; v3 closes every discovered case, produces no clean false positive, and adds no trigger on 3,996 evaluable frozen outputs. That is useful hardening evidence, but it does not prove robustness to unseen semantic attacks or new application domains.

External benchmark validation remains partial. The hosted and local benchmark runs now provide more native model/tool evidence, but they still do not constitute direct external validation of NIYAMTRACE-X because the external runners bypass the semantic authorization layer. In the expanded T4 run, Granite and Qwen each yield nine valid AgentDojo cases across banking, workspace, and travel; Slack fails for both. Granite completes six local τ^3 trajectories across airline, retail, and telecom, while Qwen completes four and exceeds the 8,192-token context on both telecom tasks. BFCL times out for both models without an official retained score. The independent Llama-family continuation does not reach inference because of a notebook-level `HF_TOKEN` initialization error. We therefore do

not claim BFCL performance, complete AgentDojo coverage, three-family external generalization, or system-level external superiority. Direct external validation requires integrating the NiyamTrace-X authorization decision into the benchmark action path rather than evaluating the base model interface alone.

Execution boundary. The corrected reference broker fails closed across the tested replay, tampering, expiry, identity, policy-change, stale-snapshot, predicted-target, and crash-before-commit conditions, and it closes the phantom-row ordering flaw found in the earlier design. Those results do *not* show that the audited prototype enforces the same properties end to end. The remaining systems task is integration: production execution must require capability verification, provider-side idempotency, restart recovery, and independent measurement of the actual effect.

Statistical scope. Zero observed unsafe ALLOW decisions is not the same as zero true risk. Under an independent-opportunity interpretation, the case-level one-sided 95% upper bound is about 0.20%. Using semantic groups as the sampling unit gives a more conservative bound of about 0.80% for the probability that an eligible group contains any unsafe ALLOW. The recovered per-case files also support group-clustered bootstrap intervals, which we report separately from case-level exact intervals. All of these quantities still describe an internally generated holdout and should not be read as guarantees for an external population.

11. Future Work

The highest-priority experiment is now a *direct* external integration rather than another model-only benchmark sweep. BFCL and AgentDojo provide published function-calling and adversarial tool-use testbeds [12, 2], while τ -bench provides a published stateful tool-agent-user setting [13]. In the next study, every candidate benchmark action should pass through the NiyamTrace-X semantic contract, Anchor Lock, safe-core decision, and—for effectful actions—the execution broker. Native benchmark metrics should be reported alongside unsafe-authorization, effect-broadening, clarification, and fail-closed metrics, without forcing incompatible benchmarks into a single score. Multilingual stress cases should also be aligned with the failure modes documented by Lost in Execution [6].

The model-only local study still identifies practical requirements for that integration: BFCL needs enough runtime to retain its official score; AgentDojo Slack requires a successful native slice; long stateful prompts need a context budget that leaves room for generation; and the third independent model family must complete inference rather than fail during setup. A three-family direct external study, with fixed prompts, tool schemas, temperatures, and retry budgets, is the threshold we would require before strengthening the current generalization claim.

On the systems side, Verified Execution Broker V2 should move from an isolated reference design into the public runtime

so that the legacy executor cannot be called without a gate-issued capability. The production path should bind actor, tool, argument hash, authorized targets and fields, predicted effect, state snapshot, policy hash, expiry, and nonce, and should enforce

Replay $\Rightarrow$ NoSecondEffect, StaleSnapshot $\Rightarrow$ NoCommit,

ActualDelta = PredictedDelta

or fail closed and roll back. Provider idempotency, crash/restart recovery, and non-bypassable deployment checks are still required before the scope of the paper can move from semantic authorization to complete execution mediation.

A separate exploratory group-split risk-calibration layer removes 26 unsafe raw ALLOW decisions on one 600-case held-out split while preserving 85.6% of raw auto-allow utility. The result is not stable enough to act as a safety boundary: 5 of 20 repeated group splits still contain at least one unsafe ALLOW. We therefore treat selective risk scoring as an escalation signal for distribution shift, not as a substitute for the deterministic runtime.

12. Ethical Considerations

The benchmark is designed to study authorization safety rather than automate harmful activity, and the runtime audit uses only a synthetic in-memory ERP. We report implementation weaknesses with deliberately bounded language and do not present an internal API bypass as a demonstrated remote exploit. The language coverage is also incomplete: model outputs and benchmark decisions may reflect linguistic or cultural bias, so broader language evaluation and human review remain necessary before use in high-stakes settings.

Data and Code Availability

The implementation is available at <https://github.com/bnssaanirudh/NiyamTrace-X>. The repository audit reported in this paper is bound to commit c14661dbd11c42ebd1019b6a1a5c49b8643da137. The frozen holdout, per-case result hashes, statistical recomputation scripts, post-freeze hardening outputs, reference-broker results, and the two local external-run archives are retained as reproducibility artifacts. The accompanying reproducibility manifest records complete SHA-256 values for both local result archives. Model-weight files are excluded because they are recoverable from their pinned upstream checkpoints.

13. Conclusion

NIYAMTRACE-X is built around a conservative idea: when a security-relevant instruction admits several plausible meanings, the agent’s executable authority should shrink to the effects those meanings share. The frozen v2 runtime implements that idea through set-valued interpretation, effect-language intersection, independent semantic anchoring, and clarification recovery placed downstream of the authority constraint.

On the 2,000-case group-disjoint multilingual holdout, Qwen3.5-122B reaches 99.85% final decision accuracy after all 80 observed raw unsafe ALLOW decisions are intercepted. GPT-OSS-120B reaches 99.80% after both of its observed raw unsafe ALLOW decisions are intercepted. Both runs end with zero observed unsafe ALLOW decisions and zero false blocks, and the group-clustered intervals over 500 semantic groups remain high. The frozen-log ablation clarifies why: Anchor Lock supplies the security gain, while clarification recovery is safe only in the intended composition after anchoring. The post-freeze metamorphic campaign then exposes, rather than hides, a scope-broadening weakness; Anchor Lock v3 closes all 1,496 discovered misses, reaching 100% recall on 10,248 critical attacks with zero clean false positives and no new trigger on any evaluable frozen output. A separate transaction-first reference broker also passes 1,320 execution scenarios, although it is not yet integrated as a non-bypassable production path.

Taken together, the evidence supports a strong but deliberately bounded conclusion. On the evaluated internal domain, set-valued semantic-to-effect authorization substantially separates imperfect multilingual interpretation from unsafe authorization. The frozen-log ablation and post-freeze metamorphic campaign show where that gain comes from and where the original guard failed; the transaction-first reference broker separately strengthens the execution design. The new local runs add useful evidence about model/tool compatibility: Granite and Qwen produce native AgentDojo and stateful τ^3 outputs under the T4 harness, but BFCL does not retain a score and the independent Llama-family continuation fails before inference because of an infrastructure bug. More importantly, those external runners bypass NiyamTrace-X itself. Direct benchmark integration and production-grade authorization-to-commit mediation therefore remain the next decisive tests, and three-family external generalization is not claimed.

References

- [1] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the Risks of LM Agents with an LM-Emulated Sandbox,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” in *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024, doi: 10.52202/079017-2636.
- [3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjectAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024, doi: 10.18653/v1/2024.findings-acl.624.
- [4] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [5] Z. Zhang, S. Cui, Y. Lu, J. Zhou, J. Yang, H. Wang, and M. Huang, “Agent-SafetyBench: Evaluating the Safety of LLM Agents,” in *Second Conference on Language Modeling*, 2025.
- [6] Z. Luo, T. P. Kutralingam, O. N. Okoani, W. Xu, H. Wei, and X. Hu, “Lost in Execution: On the Multilingual Robustness of Tool Calling in Large Language Models,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 44059–44077, 2026, doi: 10.18653/v1/2026.acl-long.2039.
- [7] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems 36*, pp. 68539–68551, 2023, doi: 10.52202/075280-2997.
- [8] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, “API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023, doi: 10.18653/v1/2023.emnlp-main.187.
- [9] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [10] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” in *Advances in Neural Information Processing Systems 37*, pp. 126544–126565, 2024, doi: 10.52202/079017-4020.
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [12] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stolica, and J. E. Gonzalez, “The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models,” in *Proceedings of the 42nd International Conference on Machine Learning*, PMLR, vol. 267, pp. 48371–48392, 2025.
- [13] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A Benchmark for Tool-Agent-User Interaction in

Real-World Domains,” in *The Thirteenth International Conference on Learning Representations*, 2025.

- [14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90, 2023, doi: 10.1145/3605764.3623985.
- [15] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, “AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations,” in *35th USENIX Security Symposium (USENIX Security 26)*, pp. 1547–1566, 2026.
- [16] J. Han, W. Buntine, and E. Shareghi, “Towards Uncertainty-Aware Language Agent,” in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 6662–6685, 2024, doi: 10.18653/v1/2024.findings-acl.398.
- [17] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.
- [18] B. W. Lampson, “A Note on the Confinement Problem,” *Communications of the ACM*, vol. 16, no. 10, pp. 613–615, 1973, doi: 10.1145/362375.362389.
- [19] D. D. Clark and D. R. Wilson, “A Comparison of Commercial and Military Computer Security Policies,” in *1987 IEEE Symposium on Security and Privacy*, pp. 184–195, 1987, doi: 10.1109/SP.1987.10001.
- [20] F. B. Schneider, “Enforceable Security Policies,” *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000, doi: 10.1145/353323.353382.
- [21] J. Ligatti, L. Bauer, and D. Walker, “Edit Automata: Enforcement Mechanisms for Run-Time Security Policies,” *International Journal of Information Security*, vol. 4, no. 1–2, pp. 2–16, 2005, doi: 10.1007/s10207-004-0046-8.